IoT 1

Les 8 – WiFi en MQTT

MQTT

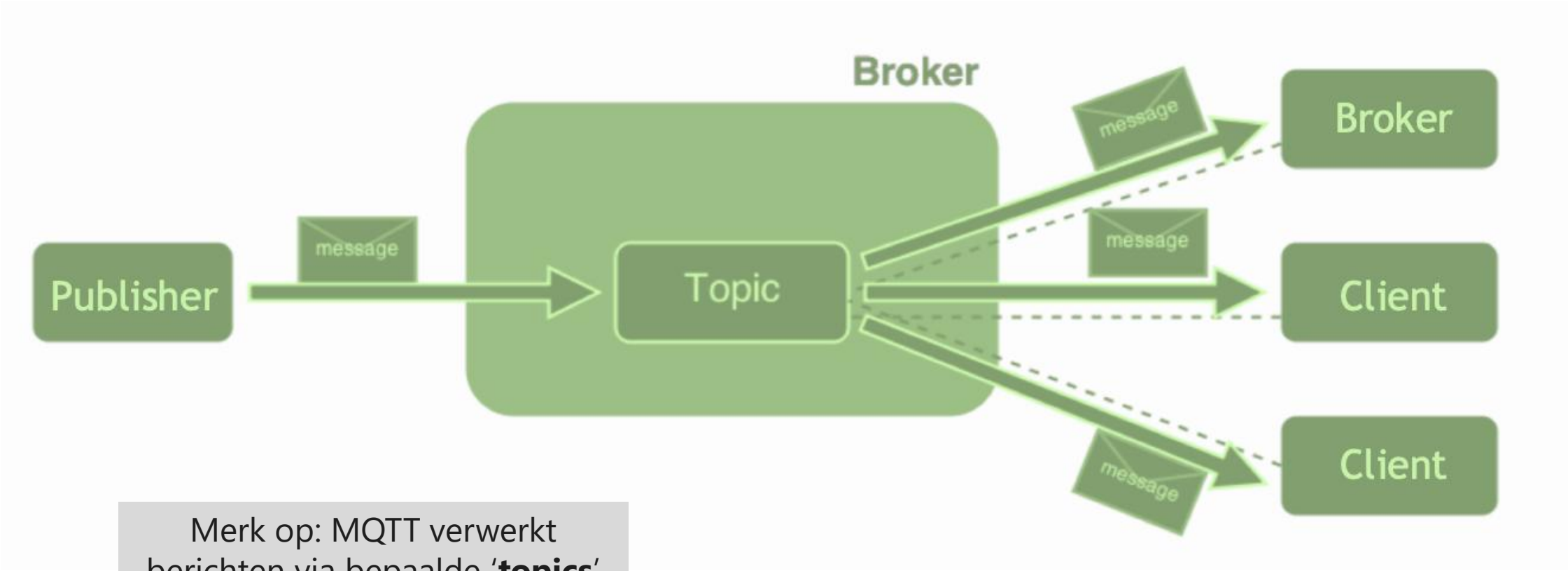
MQTT

- **Wat**?
  - MQTT (**Message Queue Telemetry Transport**) is an ISO standard (ISO/IEC PRF 20922) **publish-subscribe-based messaging protocol**.
  - It works **on top of** the **TCP/IP** protocol.
  - It is designed for connections with remote locations where a "**small code footprint**" is required or the network bandwidth is limited.
  - The publish-subscribe messaging pattern **requires a message broker**.

Bron: https://en.wikipedia.org/wiki/MQTT

hogeschool vives

MQTT

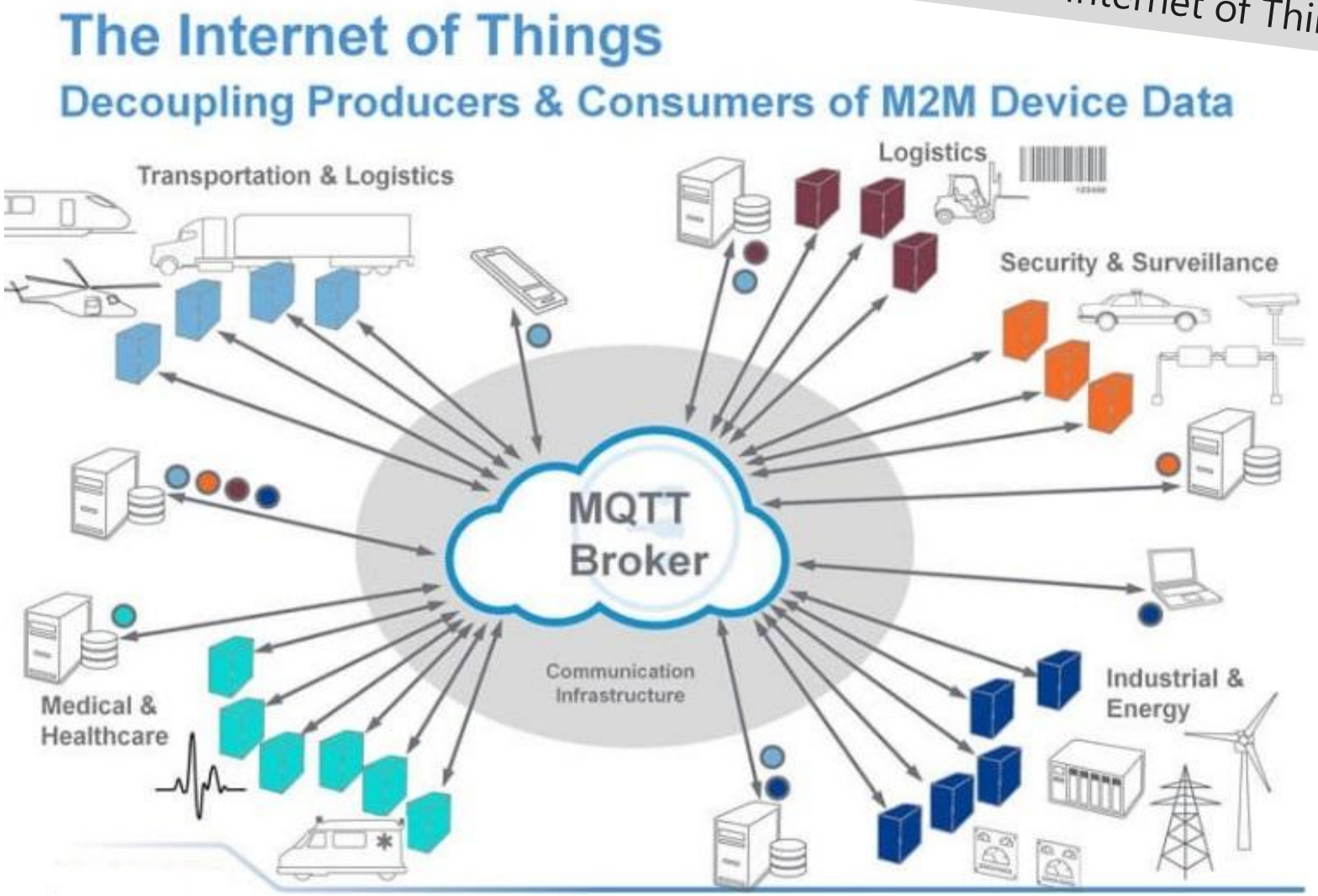


Bron: https://networkridge.com/wp-content/uploads/2020/08/image-1.png

Merk op: MQTT verwerkt berichten via bepaalde '**topics**' (onderwerpen)! Daarop kan je **abonneren** of **publiceren**.

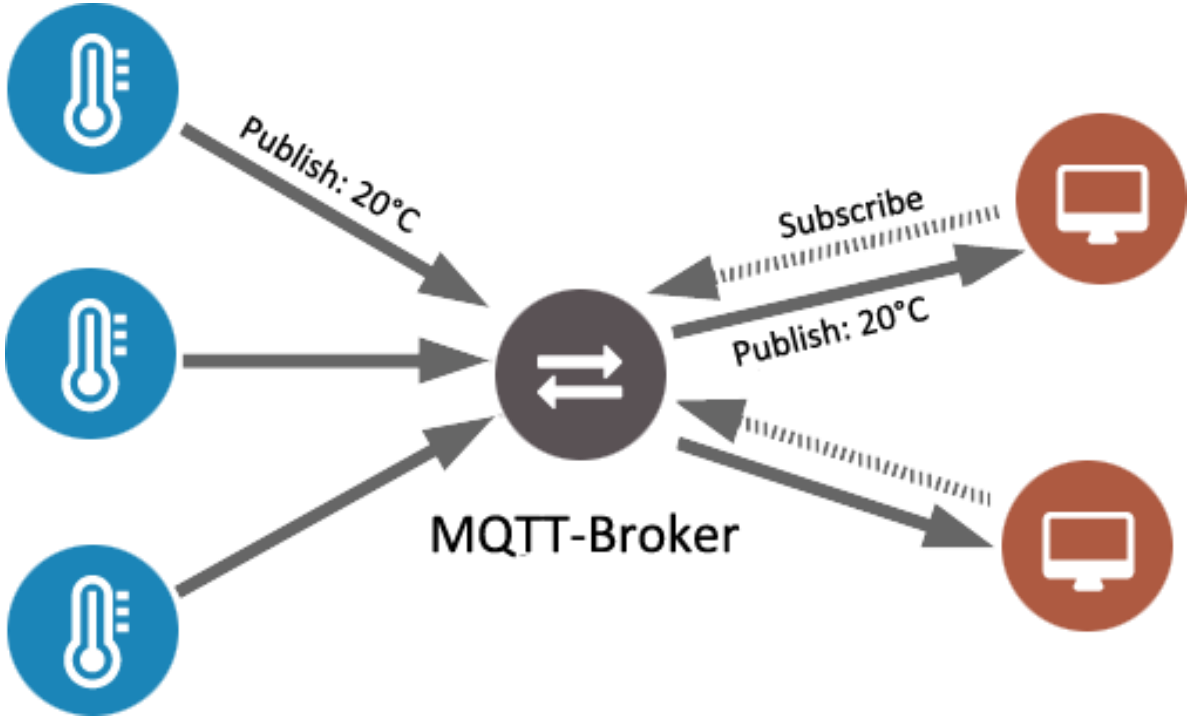
MQTT

IoT = Internet of Things!!!!



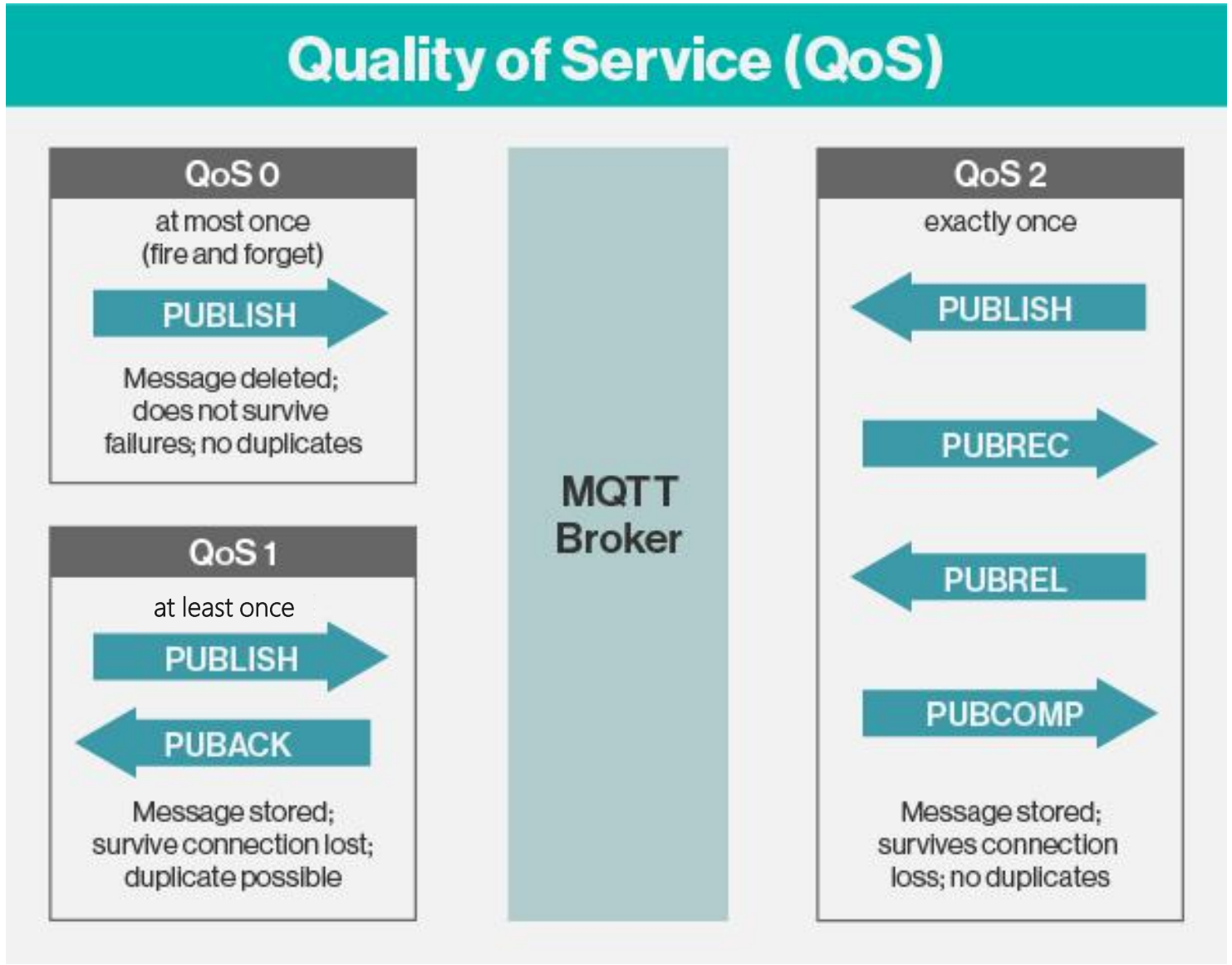
Bron: https://www.linkedin.com/pulse/industry-40-interoperability-internet-of-things-maximiliano/

MQTT



Bron: https://www.researchgate.net/

MQTT



PUBREC = publish received
PUBREL = publish release
PUBCOMP = publish complete

Bron: https://www.navixy.com/wp-content/uploads/2021/08/chrome_q1ak4noiok-600x466.png

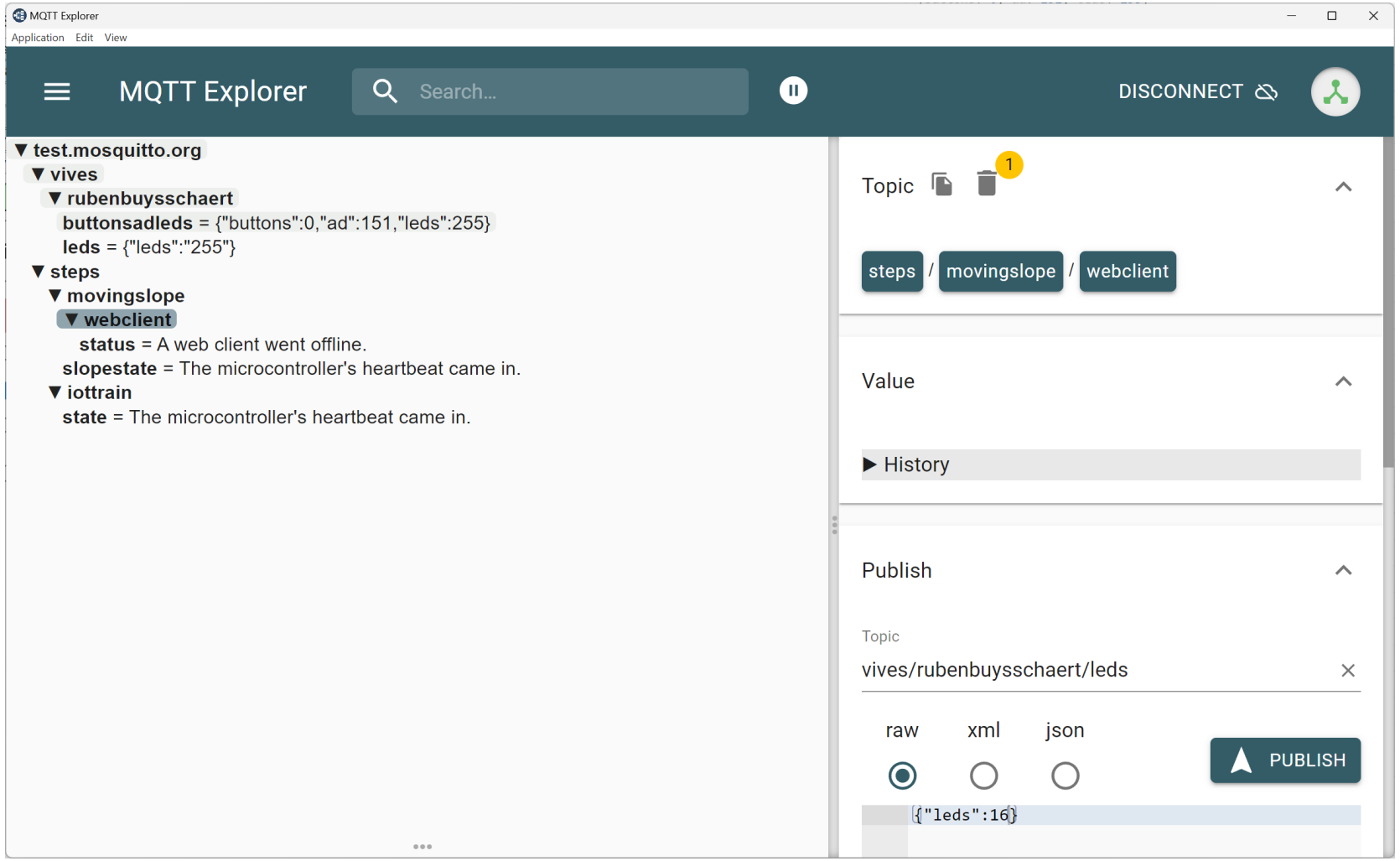
MQTT

- **Broker**
  - Een message broker is dus een **stuk software** dat berichten kan ontvangen en doorsturen naar de geïnteresseerde gebruikers (zie voorgaande afbeeldingen).
  - Je kan **zelf** je MQTT-broker opzetten (mosquitto.org) **of** gebruik maken van **bestaande** brokers (test.mosquitto.org of mqtt.rubu.be of …).
  - Dat kan op je eigen laptop, op een cloud server,op een **Raspberry Pi**, …

MQTT

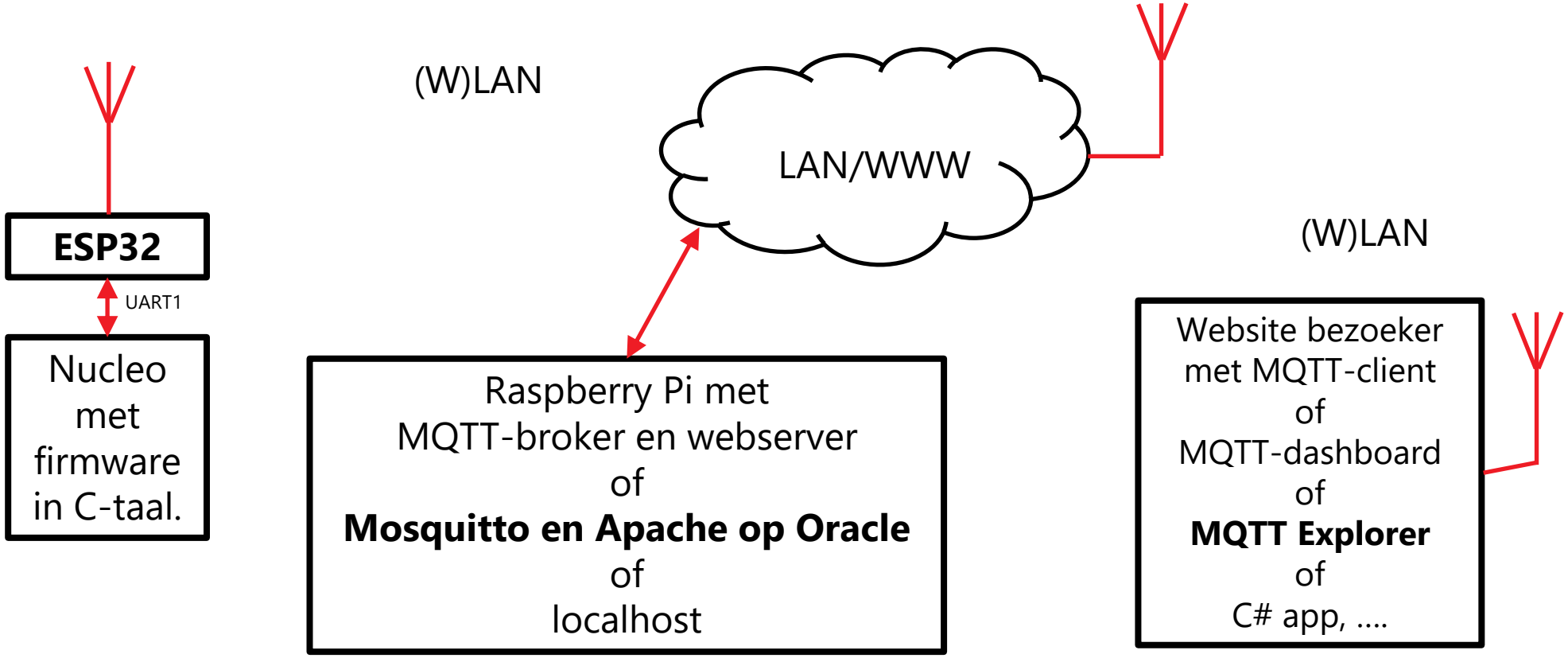
- **Client**
  - Een ‘gebruiker’ van het MQTT-protocol is een ‘**client**’. Zo’n client kan berichten verzenden (**publish**) of ontvangen (**subscribe**).
  - Er zijn **tal van mogelijkheden** om MQTT te implementeren in jouw software. Onder andere voor: **C#**, **JavaScript**, Python, C, …
  - Je kan echter ook gebruik maken van bestaande software om berichtjes te volgen, bijvoorbeeld: **MQTT Explorer**… (https://mqtt-explorer.com/)

MQTT


MQTT Explorer
Application Edit View
MQTT Explorer
Search...
DISCONNECT
test.mosquitto.org
vives
rubenbuysschaert
buttonsadleds = {"buttons":0,"ad":151,"leds":255}
leds = {"leds":"255"}
steps
movingslope
webclient
status = A web client went offline.
slopestate = The microcontroller's heartbeat came in.
iottrain
state = The microcontroller's heartbeat came in.
Topic
1
steps / movingslope / webclient
Value
History
Publish
Topic
vives/rubenbuysschaert/leds
raw
xml
json
PUBLISH
{"leds":16}

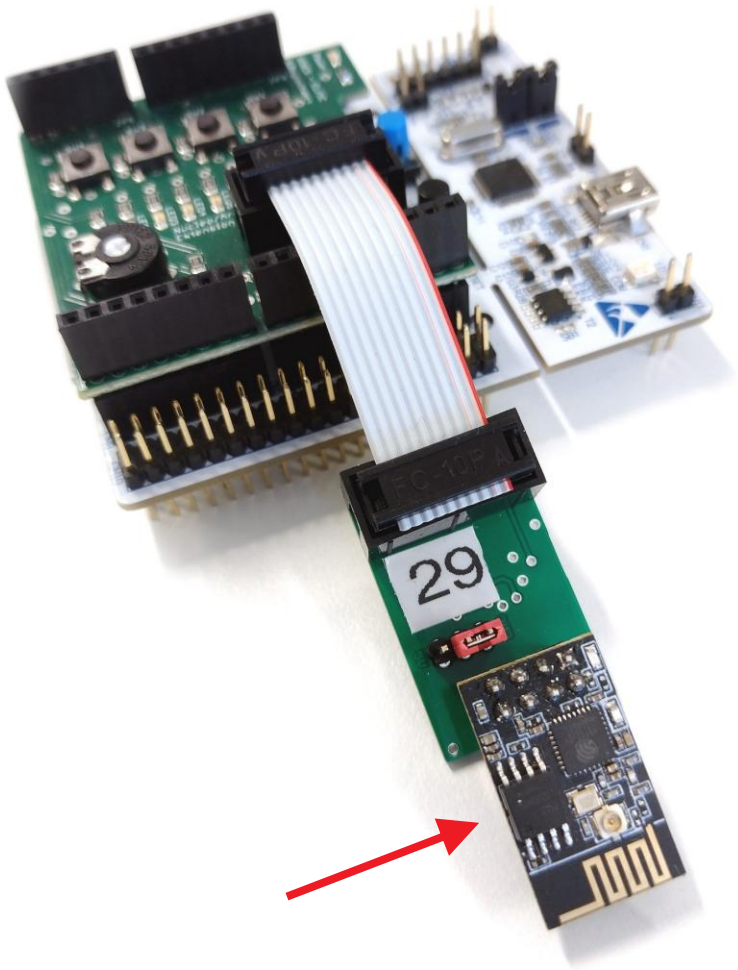
MQTT

- **Hoe**?


(W)LAN
LAN/WWW
(W)LAN
ESP32
UART1
Nucleo met firmware in C-taal.
Raspberry Pi met MQTT-broker en webserver of **Mosquitto en Apache op Oracle** of localhost
Website bezoeker met MQTT-client of MQTT-dashboard of **MQTT Explorer** of C# app, ...

MQTT

- **Hardware**
  - We maken gebruik van een **ESP32 C3**-module.
  - Die kan gekoppeld worden via de **10-pin header** op het Nucleo Extension Shield, via **USART1**.
  - De module is herprogrammeerd met een **AT-firmware** (gewisselde RX TX pinnen).



MQTT

COM22 - PuTTY

```
AT+GMR
AT version:3.1.0.0-dev(s-7a56e30 - ESP32C3 - Apr 11 2023 09:48:35)
SDK version:v5.0-541-g885e501d99
compile time(dfa2d23):Apr 13 2023 16:01:17
Bin version:2.5.0(MINI-1)

OK
AT+CIPSTAMAC?
+CIPSTAMAC:"60:55:f9:7f:95:7c"

OK
AT+SYSSTORE=0

OK
AT+CWAUTOCONN=0

OK
AT+CWMODE=1,0

OK
AT+CWRECONNCFG=10,100
```

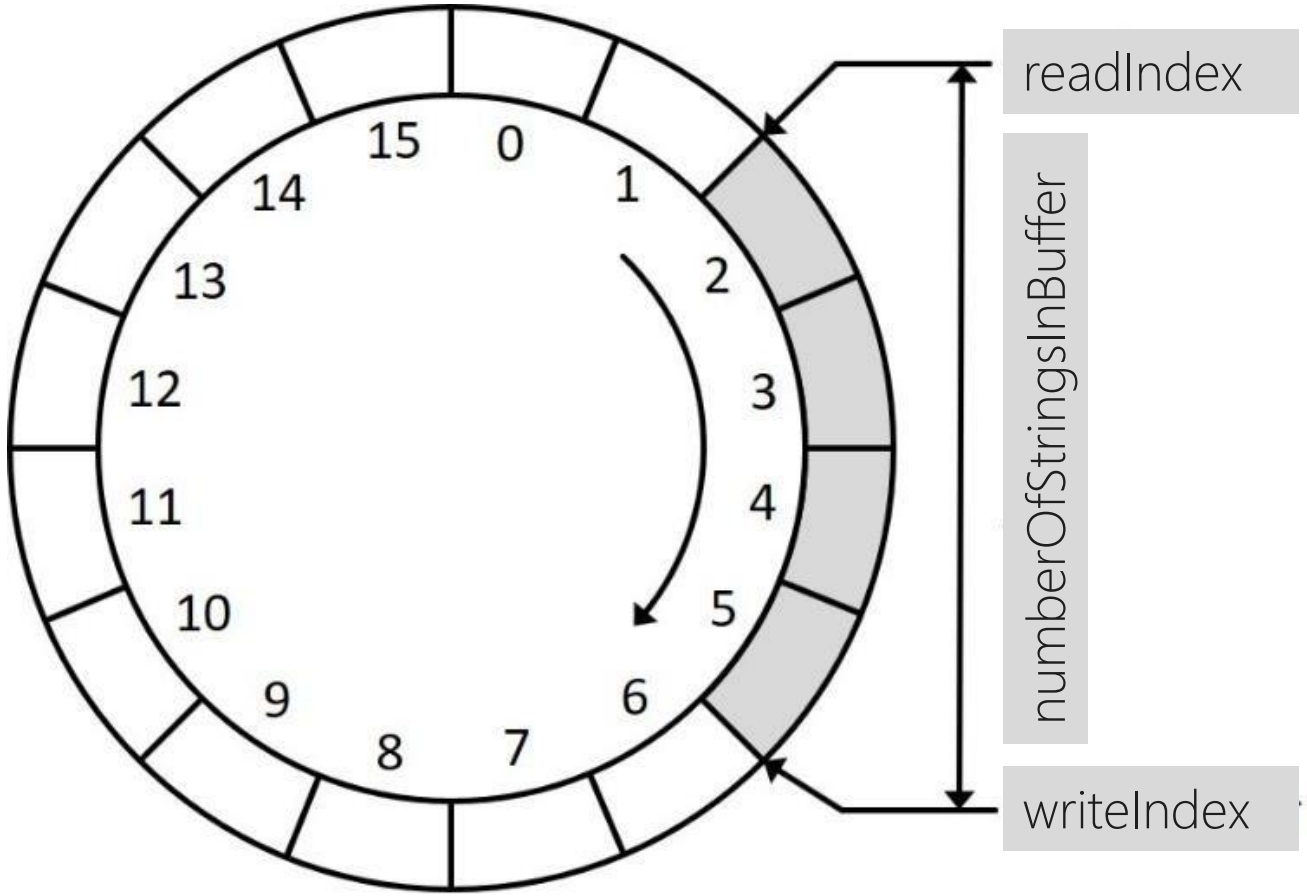
Dit is een voorbeeld van hoe met **AT-commando's** gewerkt wordt.

Hoe kan je zoiets best **aanpakken**? Mogelijke 'valkuilen'?

MQTT

- **Firmware** (voor de Nucleo)
  - De ESP32-module, communiceert dus via **AT-commando's** (via **USART1** doorgestuurd).
  - De **uitdaging** is om alle AT-commando's goed te ontvangen en tijdig te verwerken. Gebruik daarvoor een 'circulair buffer'.
  - Omdat dat buffer 'ciruclair' is, een soort cirkel van bytes dus, kan je '**oneindig lang**' gegevens opslaan. Zolang je maar snel genoeg de opgeslagen data verwerkt.
  - De '**kop**' mag de '**staart**' niet inhalen…!

MQTT



Bron: https://www.researchgate.net/

MQTT

- Bekijk de **bestanden** ‘circularbuffer.h’ en ‘circularbuffer.c’ en zoek uit hoe alles werkt…

```
// Struct die een circulair buffer voorstelt.
typedef struct
{
    uint16_t readIndex;
    uint16_t writeIndex;
    uint16_t numberOfStringsInBuffer;
    uint16_t startIndexOfCurrentlyArrivingString;
    char bufferData[CIRCULAR_BUFFER_SIZE];
}CircularBuffer;
```

ESP32-C3 demo

ESP32-C3 demo

- Aan de slag?!
  - Er is een **sjabloon** beschikbaar met als naam: “WiFi - ESP32C3 – Template – no credentials”.
  - Daarin zit een **raamwerk** om met de WiFi-modules aan de slag te gaan. **Gebruik dat** voor een nieuw project.
  - De **logingegevens** voor de Firebase database of andere servers, zijn wel verwijderd. Later maak je zelf zo’n database aan en kan je jouw eigen logingegevens toevoegen.

ESP32-C3 demo

- **IO** to UART with ESP32 C3 – **JSON**

  - Zou je alle **info** van het **Nucleo Extension Shield** (Buttons, LED's en AD), in **JSON-formaat** naar een **MQTT-broker** kunnen sturen?
  - Kan je die info dan **monitoren** met **MQTT Explorer**, een **website** en een **C# WPF** App?
  - Gebruik als **topic**:

    `achternaamvoornaam/leds` en

    `achternaamvoornaam/buttonsadleds`.

hogeschool vives

ESP32-C3 demo

- **IO** to UART with ESP32 C3 – **JSON**

```
111     // Met behulp van de User Button het automatisch publish'n in- of uitschakelen.
112     if(timeBasedPublishing && timerTicked)
113     {
114       // Timer verwerken.
115       timerTicked = false;
116
117       // MQTT-bericht publish'n.
118       sprintf(text, "{\"buttons\":%d,\"ad\":%d,\"leds\":%d}", buttonsValue, adValue, leds);
119       MqttPublish(MQTT_PUB_TOPIC, text);
120     }
```

JSON-opbouw…

OPM: gebruik de **watchdog** timer om automatisch te resetten als de hoofdlus te lang geblokkeerd staat.

Dat kan voorvallen als de AT-commando's niet tijdig/correct verwerkt geraken.

ESP32-C3 demo

- **IO** to UART with ESP32 C3 – **JSON**

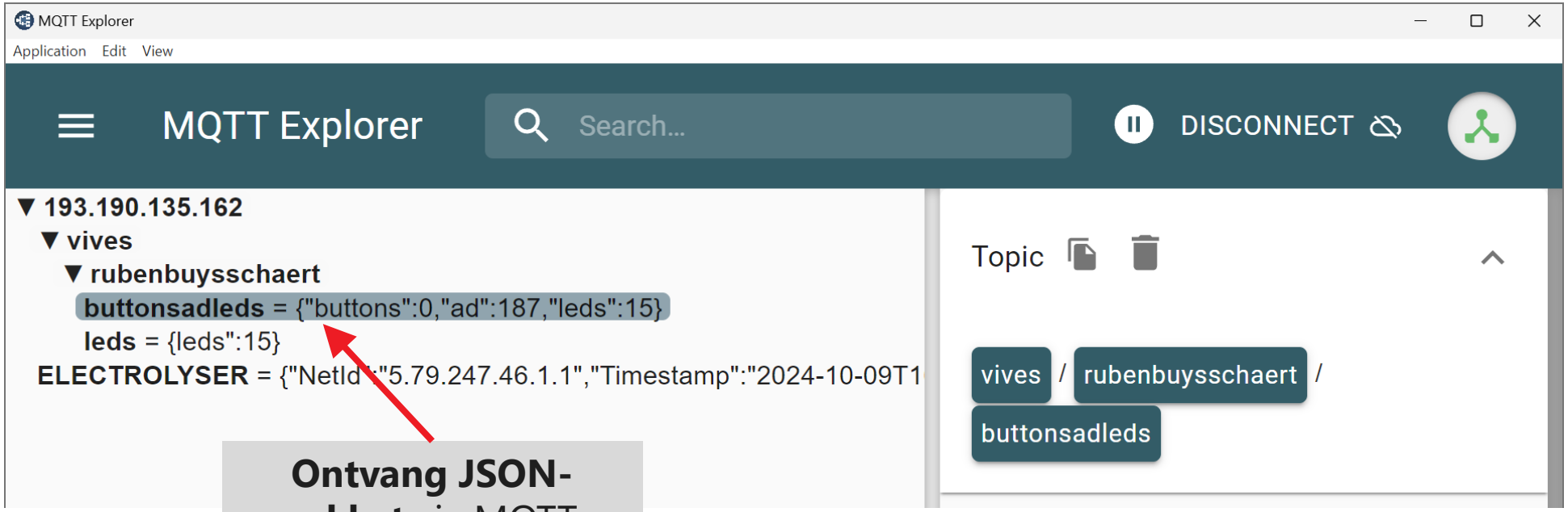
```
142     // Mogelijks ontvangen TCP-, HTTP- of MQTT-berichten zoeken in het circulair buffer.
143     // De berichten zijn van de vorm: +IPD, of +MQTTSUBRECV:0,"rubu/esp8266",3,254
144     // of +MQTTSUBRECV:0,"vives/rubenbuysschaert/leds",12,{"leds":129}
145     while(circularBuffer.numberOfStringsInBuffer > 0)
146     {
147       // Eén string ophalen.
148       circularBufferActionResult = PopStringFromCircularBuffer(&circularBuffer, receivedText);
149       if(circularBufferActionResult == readStringSucceeded)
150       {
151         // MQTT-ontvangst onderzoeken. Zoek achter "+MQTTSUBRECV:0,"het huidige topic",". Dat ge
152         sprintf(text, "+MQTTSUBRECV:0,\"%s\",", MQTT_SUB_TOPIC);
153         index = strlen(text);                                  // Opzoeken hoe lang dat gedeelte
154
155         // Begin van de lijn vergelijken met de verwachte tekst en topic.
156         if(strncmp(text, receivedText, index) == 0)
157         {
158           // Zoek uit hoeveel bytes er ontvangen werden.
159           numberOfReceivedBytes = (uint16_t)atoi(&receivedText[index]);
```

MQTT\Firmware\IO to UART with ESP32 C3 – JSON

ESP32-C3 demo

- **IO** to UART with ESP32 C3 – **JSON**

MQTT Explorer

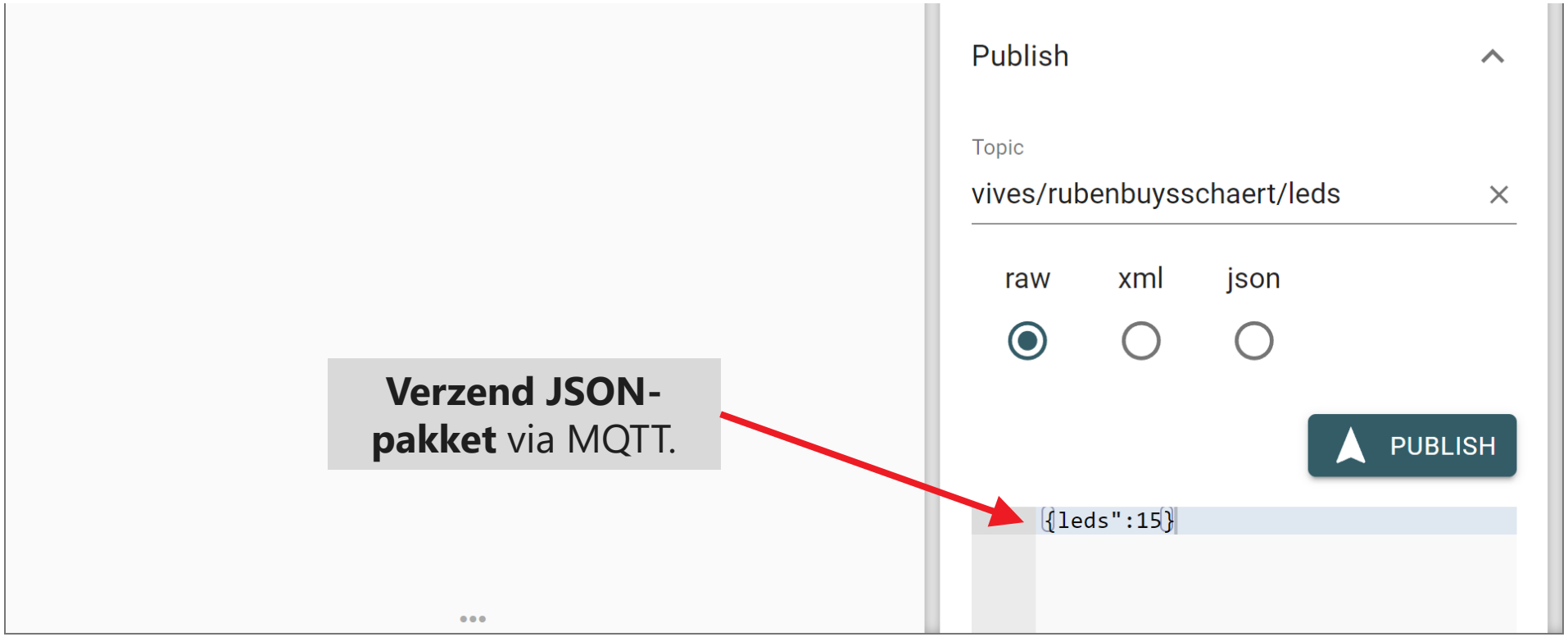


Ontvang JSON-pakket via MQTT.

Bron: https://mqtt-explorer.com/

ESP32-C3 demo

- **IO** to UART with ESP32 C3 – **JSON**



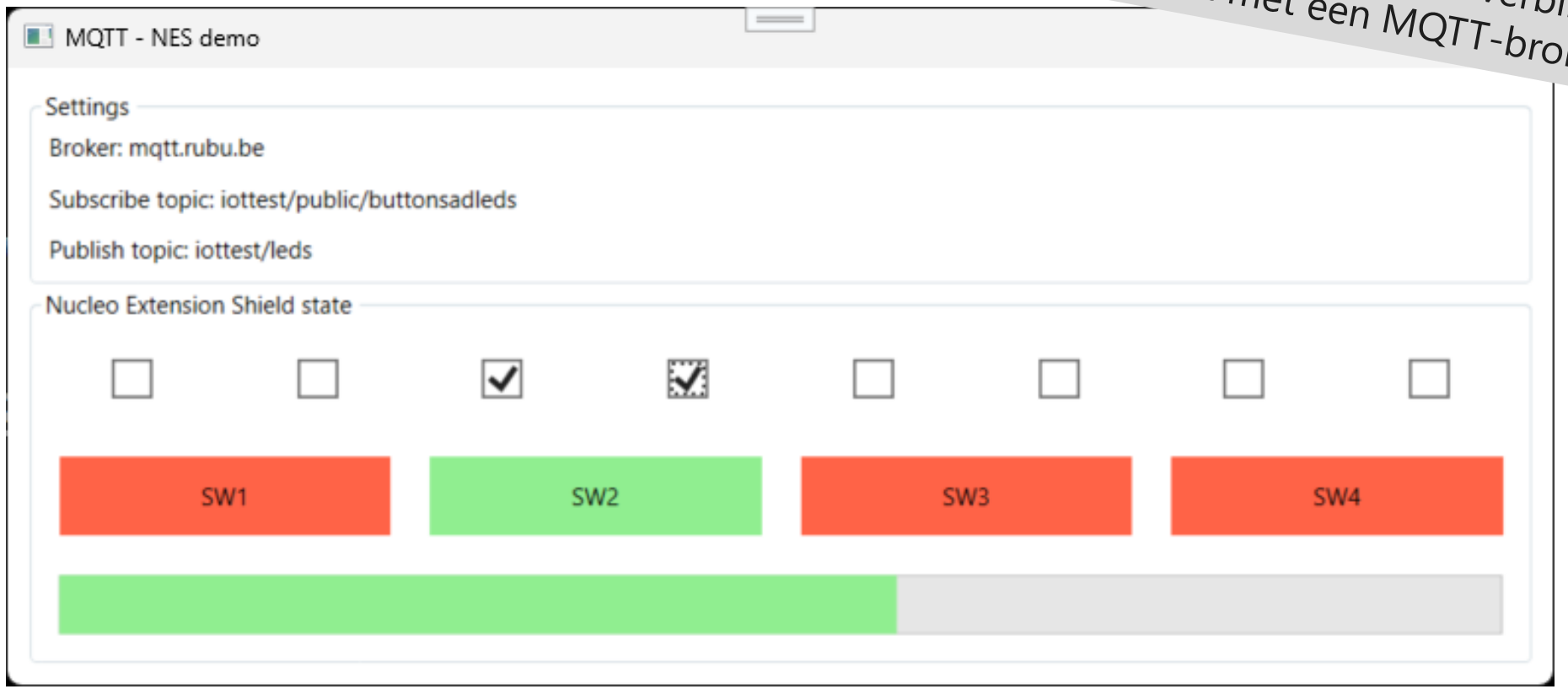
Verzend JSON-pakket via MQTT.

Bron: https://mqtt-explorer.com/

ESP32-C3 demo

- **IO** to UART with ESP32 C3 – **JSON**

C# **WPF App** die met de MQTTnet bibliotheek verbinding maakt met een MQTT-broker...

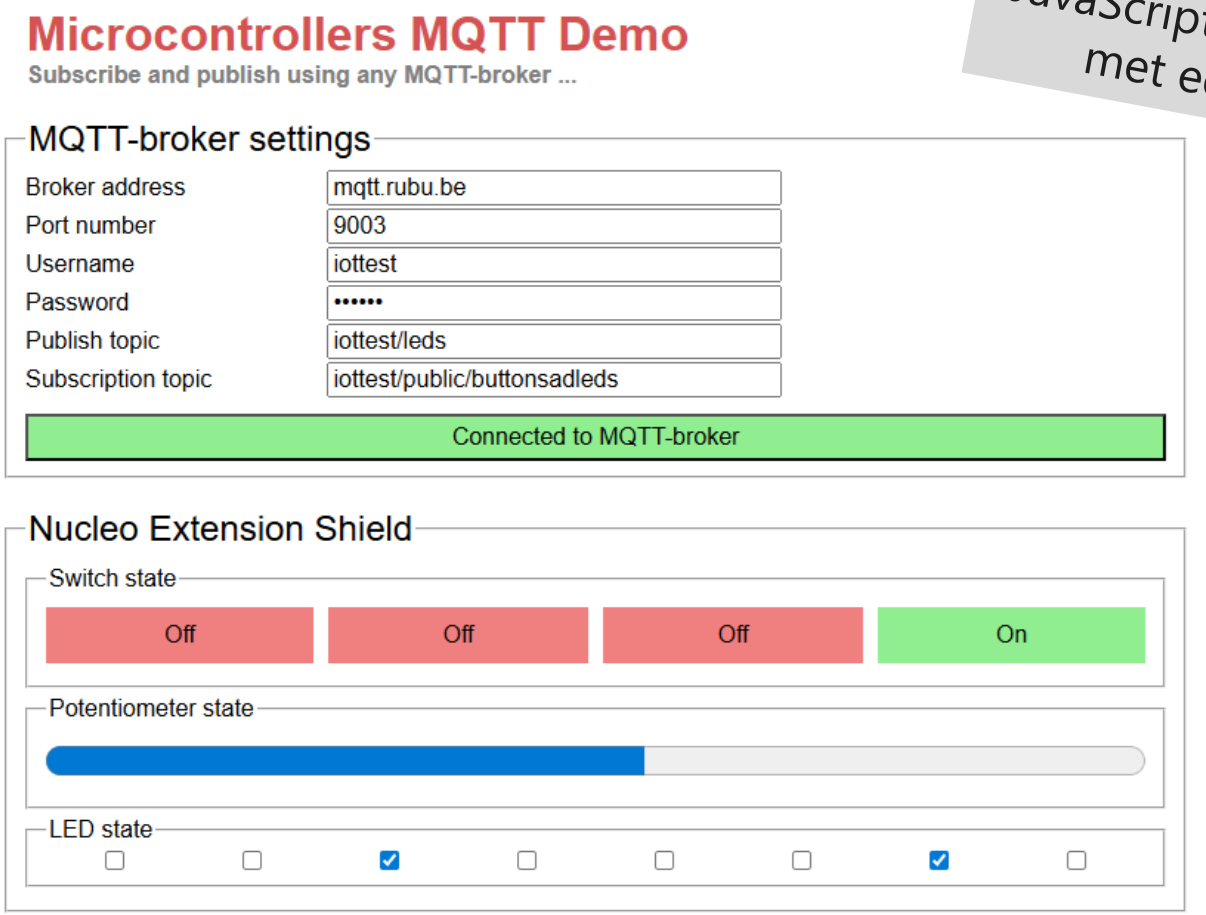


MQTT\Software\MqttNet V1

ESP32-C3 demo

- **IO** to UART with ESP32 C3 – **JSON**

Website met HTML, CSS en JavaScript die verbinding maakt met een MQTT-broker…

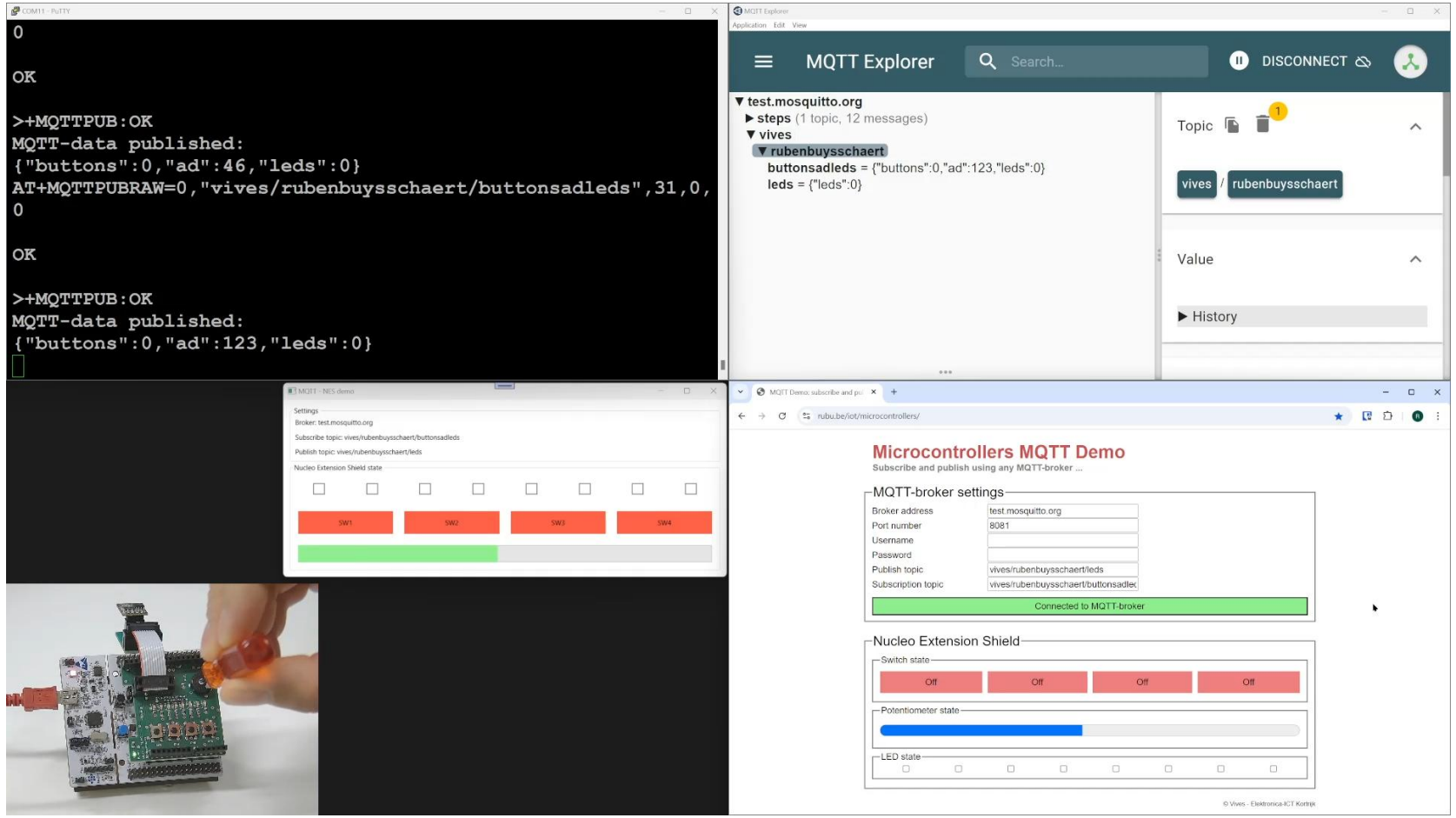


Bron: https://www.rubu.be/iot/microcontrollers/

ESP32-C3 demo

- **IO** to UART with ESP32 C3 – **JSON**

Demo(video)
all-in-one…



Bron: MQTT - NES - JSON - CS - Web.mp4

Your turn now!

- Is er nog **wat resterende** tijd?
    - Vragen?
    - Feedback voorbije theorieopdrachten?
    - Ga **zelf aan de slag** om je WiFi-module online te krijgen. Verstuur een pakketje dat via **MQTT**…
    - Samen opdrachten/oefeningen maken?